

Máquina de bolitas

Madina ha inventado una máquina de bolitas para entretener a los participantes de la IOI. La estructura interna de la máquina es la siguiente:

- La máquina consta de **nodos** N , indexados desde 0 hasta $N - 1$. El nodo $N - 1$ se denomina **raíz**.
- Para cada nodo u con $0 \leq u < N - 1$, el **padre** del nodo u es un nodo $P[u]$ con $P[u] > u$, y el nodo u es un **hijo** de $P[u]$. La raíz no tiene padre. Un nodo sin hijos se denomina **hoja**.

No conoces N ni tampoco el nodo padre $P[u]$ de ningún nodo u . En cambio, Madina te indica M , el número de hojas, y que los índices de las hojas son $0, 1, \dots, M - 1$. Tu tarea consiste en determinar la estructura interna de la máquina usándola.

Cada nodo de la máquina puede contener como máximo una **bolita**, y cada bolita tiene un **valor** que es un número entero no negativo. Decimos que un nodo tiene valor x si contiene una bolita con valor x . Un nodo está **ocupado** si contiene una bolita; en caso contrario, está **libre**. Inicialmente, todos los nodos están libres.

Puedes realizar dos tipos de operaciones:

- **insert**(U, X): intenta insertar una nueva bolita con el valor X en la hoja U .
 - Si la hoja U está ocupada, la operación retorna `false` sin insertar una bolita.
 - Si la hoja U está libre, la bolita se coloca en el nodo U . Entonces la bolita se mueve de forma repetida al padre de su nodo actual siempre que ese padre esté libre. El movimiento se detiene cuando la bolita alcanza a la raíz o un nodo cuyo padre está ocupado. La operación retorna `true`. (**insert** significa "insertar")
- **collect**(): si la raíz está libre (lo que quiere decir que la máquina está vacía), `collect` retorna un arreglo vacío. En otro caso, el procedimiento recursivo `traverse` descrito abajo recolecta y almacena los valores de todas las bolitas que están actualmente en la máquina dentro un arreglo S . Inicialmente, el arreglo S está vacío y `traverse(N - 1)` es llamado. (**collect** significa "recolectar")

```

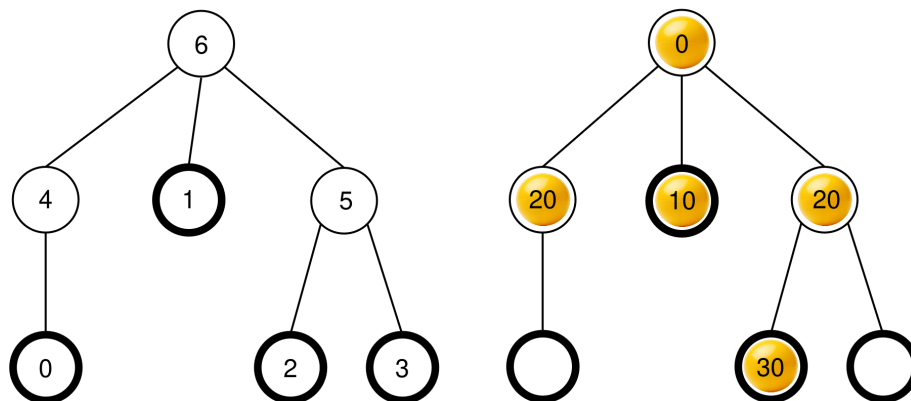
traverse(u):
    agregar el valor de la bolita en el nodo u al final de S
    sea c[u] la lista de los hijos ocupados de u
    ordenar c[u] en orden no decreciente según los valores
    de las bolitas en esos nodos (si varios nodos tienen el
    mismo valor, pueden aparecer en cualquier orden)
    para cada v en c[u]:
        traverse(v)

```

(**traverse** significa "recorrer")

Después de que esta función termina de ejecutarse, **todas las bolitas son eliminadas** de la máquina, y `collect` retorna S .

Por ejemplo, considere una máquina con $N = 7$ nodos y un arreglo de padres $P = [4, 6, 5, 5, 6, 6]$. La imagen a la izquierda muestra una máquina vacía con índices de nodos, mientras que la imagen en la derecha muestra una posible configuración de bolitas después de alguna secuencia de operaciones de tipo `insert` (esta secuencia está dada en la sección de Ejemplo).



Cuando la operación `collect()` es llamada, la raíz (nodo 6) está ocupada, entonces S es inicializada como $S = []$ y se llama a `traverse(6)`.

traverse(6): el nodo 6 tiene como valor 0; agregarlo al final de S resulta en $S = [0]$. Los hijos del nodo 6 son 4, 1, y 5, todos los cuales están ocupados. Sus valores son 20, 10, y 20, respectivamente. Ordenar de forma no decreciente resulta en $c[6] = [1, 5, 4]$ (note que $c[6] = [1, 4, 5]$ también sería válido, dado que los nodos 4 y 5 tienen el mismo valor). El procedimiento entonces llama a `traverse` en cada nodo en $c[6]$ en ese orden.

- **traverse(1):** el nodo 1 tiene como valor 10; agregarlo al final de S resulta en $S = [0, 10]$. El nodo 1 no tiene hijos ocupados, entonces esta llamada termina.
- **traverse(5):** el nodo 5 tiene como valor 20; agregarlo al final de S resulta en $S = [0, 10, 20]$. El nodo 5 tiene un hijo ocupado, el nodo 2, por lo que $c[5] = [2]$ y la función

llama a `traverse(2)`.

- **`traverse(2)`**: el nodo 2 tiene como valor 30; agregarlo al final de S resulta en $S = [0, 10, 20, 30]$. El nodo 2 no tiene hijos ocupados, entonces esta llamada termina.
- La ejecución vuelve a `traverse(5)` que ha procesado todos los hijos en $c[5]$, entonces su llamada termina.
- **`traverse(4)`**: el nodo 4 tiene el valor 20; agregarlo al final de S resulta en $S = [0, 10, 20, 30, 20]$. El nodo 4 no tiene hijos ocupados, entonces esta llamada termina.

Todas las llamadas han sido completadas y no hay más nodos que procesar. Finalmente, cada bolita es eliminada de la máquina y `collect` retorna el arreglo $S = [0, 10, 20, 30, 20]$.

Tu tarea es determinar el número de nodos N y presentar un arreglo de padres $R = [R[0], R[1], \dots, R[N-2]]$ que describe la estructura de la máquina, pero con un etiquetado potencialmente diferente con respecto a los nodos que no son hojas y no son la raíz. Formalmente, un arreglo R presentado es considerado correcto si es posible asignar una etiqueta distinta $L[u]$ con $0 \leq L[u] < N$ a cada nodo u de la máquina de tal manera que:

- $L[u] = u$ para todo $0 \leq u < M$ y $u = N-1$, y
- $L[P[u]] = R[L[u]]$ for all $0 \leq u < N-1$.

No es requerido que $R[u] > u$. La máquina **debe estar vacía** cuando presentas tu solución.

Sea K el número de operaciones `collect` realizadas, y B sea el máximo valor de cualquier bolita tomando en cuenta todas las llamadas a `insert`. Sea $C = K + B$. Entonces C **no debe exceder** 1000. Tu puntuación en algunas subtarear depende del valor de C .

Detalles de implementación

Debes implementar la siguiente función.

```
std::vector<int> find_structure(int M)
```

(**find_structure** significa "encontrar estructura")

- M : el número de hojas en la máquina.
- La función se llamada exactamente una vez por cada caso de prueba.
- La función debe retornar un arreglo $R = [R[0], R[1], \dots, R[N-2]]$ de tamaño $N-1$, un arreglo correcto de padres.

Para interactuar con la máquina, tu función puede llamar a las siguientes dos funciones.

La primera función es:

```
bool insert(int U, int X)
```

(**insert** significa "insertar")

- U : el índice de la hoja donde la bolita debería insertarse. La condición $0 \leq U < M$ debe cumplirse.
- X : el valor entero de la bolita. La condición $0 \leq X \leq 1000$ debe cumplirse.
- Esta función retorna `true` si la bolita se insertó exitosamente, o `false` si la hoja U ya estaba ocupada.
- Esta función se puede llamar un máximo de 500 000 veces.

La segunda función es:

```
std::vector<int> collect()
```

- Recolecta los valores de todas las bolitas insertadas, vacía la máquina, y retorna el arreglo almacenado S .

Si en algún momento durante la ejecución de tu programa el valor de C supera 1000 , tu solución recibirá el veredicto `Output isn't correct: Too many resources used` (que significa "La salida es incorrecta: Demasiados recursos utilizados").

La estructura de la máquina se define antes de llamar a `find_structure` . El evaluador es **determinista** en el sentido de que si se ejecuta dos veces y ambas ejecuciones realizan la misma secuencia de operaciones, las llamadas a `collect` devolverán los mismos arreglos.

Restricciones

- $2 \leq N \leq 1000$
- $1 \leq M \leq 200$
- $M < N$

Subtareas

Subtarea	Puntuación	Restricciones adicionales
1	5	La raíz tiene exactamente M hijos.
2	10	$M \leq 3$
3	25	$N \leq 200, M \leq 45$
4	60	Sin restricciones adicionales.

En las subtareas 3 y 4, tu puntuación depende en el valor de C como se indica a continuación.

Subtarea 3 (25 points)

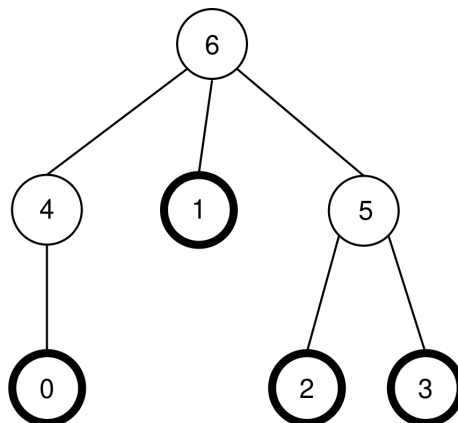
Restricciones	Puntuación
$1000 < C$	0
$45 < C \leq 1000$	13
$C \leq 45$	25

Subtarea 4 (60 points)

Restricciones	Puntuación
$1000 < C$	0
$200 < C \leq 1000$	7
$71 < C \leq 200$	$47 - \frac{C}{5}$
$44 < C \leq 71$	$104 - C$
$C \leq 44$	60

Ejemplo

Considere la misma máquina dada en la descripción, en la cual $N = 7$, $M = 4$, y $P = [4, 6, 5, 5, 6, 6]$. La máquina se muestra nuevamente en la siguiente imagen:



El evaluador realiza la siguiente llamada:

```
find_structure(4)
```

La función puede realizar la siguiente secuencia de llamadas:

1. **insert(0, 0)**: la hoja 0 está libre, por lo que una bolita con valor 0 es colocada en la hoja 0. El nodo $P[0] = 4$ está libre, por lo que la bolita se mueve al nodo 4. El nodo $P[4] = 6$ está

libre, por lo que la bola se mueve a 6. Dado que el nodo 6 es la raíz, el movimiento se detiene. La llamada retorna `true`.

2. `insert(3, 20)`: la hoja 3 está libre, entonces una bolita con valor 20 es colocada en la hoja 3. El nodo $P[3] = 5$ está libre, por lo que la bolita se mueve al nodo 5. Dado que $P[5] = 6$ está ocupado, el movimiento se detiene. La llamada retorna `true`.

3. `insert(1, 10)`: la hoja 1 está libre, por lo que una bolita con un valor de 10 es colocada en la hoja 1. Dado que $P[1] = 6$ está ocupada, la bolita se queda en el nodo 1. La llamada retorna `true`.

4. `insert(2, 30)`: la hoja 2 está libre, entonces una bolita con un valor de 30 es colocada en la hoja 2. Dado que $P[2] = 5$ está ocupada, la bolita se queda en el nodo 2. La llamada retorna `true`.

5. `insert(1, 25)`: la hoja 1 está ocupada, por lo que la bolita no es colocada en la hoja 1. La llamada retorna `false`.

6. `insert(0, 20)`: la hoja 0 está libre, por lo que una bolita con un valor de 20 es colocada en la hoja 0. El nodo $P[0] = 4$ está libre, entonces la bolita se mueve al nodo 4. Dado que $P[4] = 6$ está ocupado, el movimiento se detiene. La llamada retorna `true`.

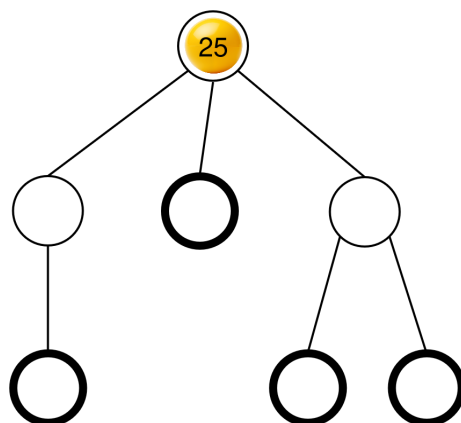
La configuración de bolitas que resulta de lo anterior ya fue mostrada en la descripción de la tarea.

Luego, la función puede llamar:

```
collect()
```

La ejecución de esta llamada ya fue explicada en la descripción de la tarea, entonces esta llamada retorna $S = [0, 10, 20, 30, 20]$. Después, la máquina vuelve a estar vacía.

Luego, la función puede llamar `insert(2, 25)`. La hoja 2 está libre, por lo que una bolita con valor 25 es colocada en la hoja 0. Esta bolita entonces se mueve al nodo 5, y luego al nodo 6, donde se detiene. La llamada retorna `true`. La configuración de bolitas que resulta de lo anterior se muestra en la siguiente imagen.



Finalmente, la función puede llamar `collect()` que retorna $S = [25]$ y vacía a la máquina.

Hay dos arreglos de padres correctos que `find_structure` puede retornar: $R = P = [4, 6, 5, 5, 6, 6]$ (que corresponde al etiquetado con $L = [0, 1, 2, 3, 4, 5, 6]$), y $R = [5, 6, 4, 4, 6, 6]$ (que corresponde al etiquetado con $L = [0, 1, 2, 3, 5, 4, 6]$). En este ejemplo, $K = 2$ y $B = 30$, por lo que $C = 32$.

Sample Grader (Evaluador de Ejemplo)

Formato de entrada:

```
N M
P[0] P[1] P[2] ... P[N-2]
```

Formato de salida:

```
K B C
Q
R[0] R[1] R[2] ... R[Q-1]
```

Aquí, Q es el tamaño del arreglo R retornado por `find_structure`.